EXPERIMENT- 3

AIM:

To design and develop a responsive, real-time online movie ticket booking system named CinePass (similar to BookMyShow) with multi-user concurrency handling, transaction countdown timer, pay-at-counter verification, and PDF receipt generation using Node.js/Express, SQLite/MySQL, WebSockets, and HTML5/Tailwind CSS.

Tasks:

- Create a user registration and login authentication interface with session management.
- Develop an interactive seat matrix categorized by tiers (VIP, Premium, Standard) with real-time seat status tracking.
- Implement a 5-minute (or 15-minute) transaction window countdown timer with automatic hold expiration without permanently freezing seats.
- Implement atomic concurrency control to handle race conditions:
  - Case 1: Single user completes booking; the seat is permanently locked with a unique confirmation code.
  - Case 2: A second user selecting an already locked/held seat receives an immediate "Seat not available" message.
  - Case 3: Two users confirming the same seat at the exact same timestamp have both requests safely rejected back to seat selection.
- Provide a single "Pay at Counter" workflow:
  - Act 1: Display on-screen booking verification code for box-office payment.
  - Act 2: Generate a downloadable PDF receipt containing movie, hall, seat, and code details.
- Provide a user dashboard to track booked tickets and counter codes.
- Implement WebSockets for instant, multi-client real-time seat synchronization across browser sessions.

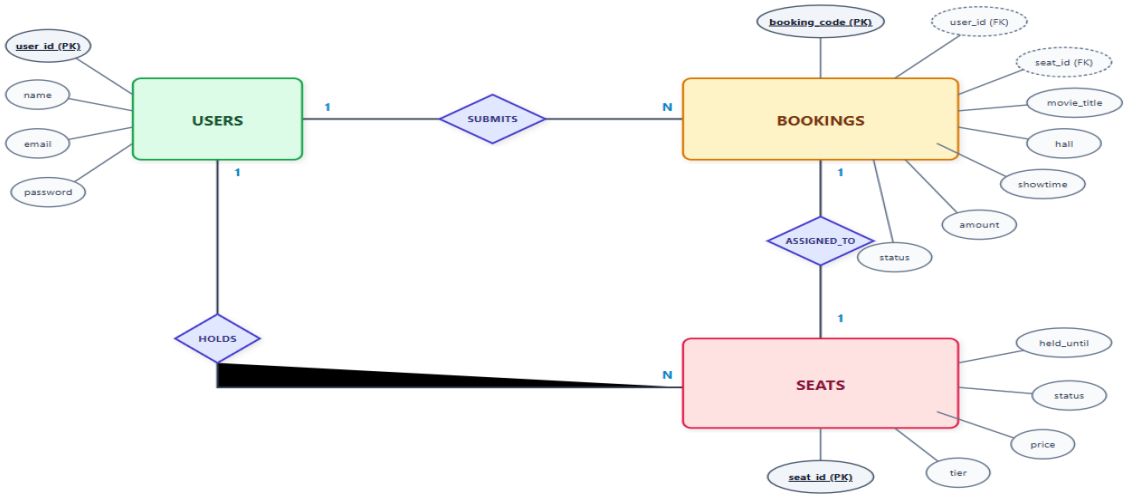
PROCEDURE:

1. Install Node.js, Express, SQLite3 (or MySQL2), WebSockets (ws), and jsPDF packages.
2. Set up the project directory structure with server.js and a public/ directory containing login.html, index.html, and dashboard.html.
3. Initialize the database schema with users, seats, and bookings tables.
4. Configure backend API routes for authentication, seat selection, atomic seat locking, cancellation, and booking confirmation.
5. Integrate a WebSocket server to broadcast seat state updates (AVAILABLE, HELD, BOOKED) in real-time.
6. Build the frontend user interface using Tailwind CSS and Lucide icons.
7. Start the server using node server.js on http://localhost:5000.

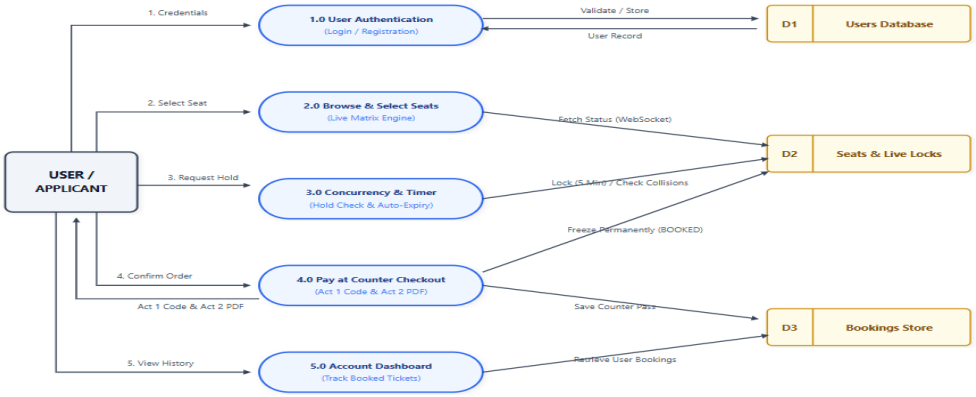
Maninder Kaur (25MCA20135)
25MCA-6 (B)

8. Open two separate browser windows (normal and incognito) to log in as two distinct users and verify real-time concurrency and seat collisions.
9. Verify that ticket confirmation and PDF receipts are accurately generated and stored in the database.

ER DIAGRAM:



DFD DIAGRAM:



CODE:

Database Schema & Connection (server.js):

```
const sqlite3 = require('sqlite3').verbose();
const db = new sqlite3.Database('cinema.db');

db.serialize(() => {
  db.run(`CREATE TABLE IF NOT EXISTS users (
```

```
    id TEXT PRIMARY KEY, name TEXT NOT NULL, email TEXT UNIQUE NOT NULL, password
TEXT NOT NULL
  )`);
  db.run(`CREATE TABLE IF NOT EXISTS seats (
    seat_id TEXT PRIMARY KEY, tier TEXT, price REAL,
    status TEXT CHECK(status IN ('AVAILABLE', 'HELD', 'BOOKED')) DEFAULT 'AVAILABLE',
    held_by TEXT, held_until INTEGER, booked_by TEXT, booking_code TEXT
  )`);
  db.run(`CREATE TABLE IF NOT EXISTS bookings (
    booking_code TEXT PRIMARY KEY, user_id TEXT, seat_id TEXT,
    movie_title TEXT, hall TEXT, showtime TEXT, amount REAL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP, status TEXT DEFAULT
'CONFIRMED_UNPAID'
  )`);
});
```

Atomic Concurrency Lock Handling (server.js):

```
app.post('/api/seats/hold', async (req, res)
=> {
  const { seatId, userId,
holdDurationSeconds = 300 } = req.body;
  const seat = await dbGet('SELECT *
FROM seats WHERE seat_id = ?',
[seatId]);
  const now = Date.now();

  // Case 2: Seat already booked or held by
another user
  if (seat.status === 'BOOKED') {
    return res.status(409).json({ success:
false, message: 'The seat is already
booked select any other seat.' });
  }
  if (seat.status === 'HELD' &&
seat.held_until > now && seat.held_by
!== userId) {
    return res.status(409).json({ success:
false, message: 'The seat is currently in
payment by another user. Select another
seat.' });
  }

  // Lock seat for transaction window
  const heldUntil = now +
(holdDurationSeconds * 1000);
```

```
  await dbRun(`UPDATE seats SET status
= 'HELD', held_by = ?, held_until = ?
WHERE seat_id = ?`, [userId, heldUntil,
seatId]);

  broadcastSeatState();
  res.json({ success: true, heldUntil });
});
```

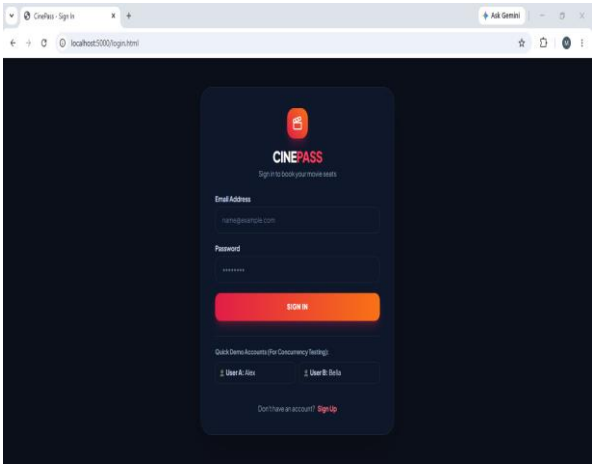
Automatic Hold Release Worker (server.js):

```
setInterval(async () => {
  const now = Date.now();
  const res = await dbRun(`
    UPDATE seats
    SET status = 'AVAILABLE', held_by
= NULL, held_until = NULL
    WHERE status = 'HELD' AND
held_until <= ?
  `, [now]);

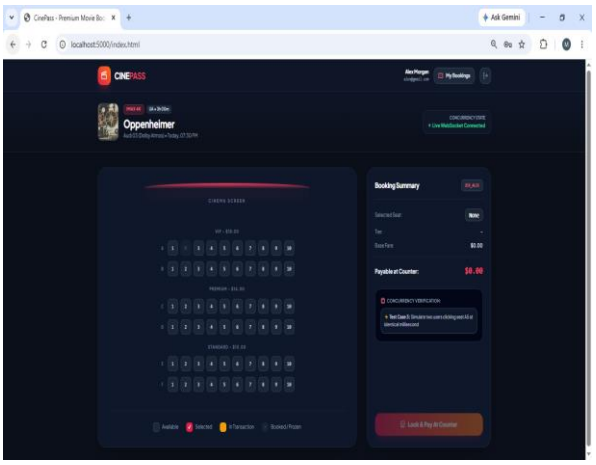
  if (res.changes > 0)
broadcastSeatState();
}, 1000);
```

SCREENSHOTS:

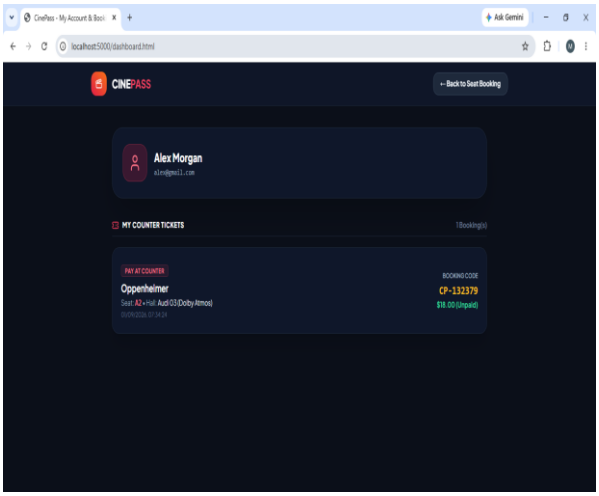
LOGIN PAGE



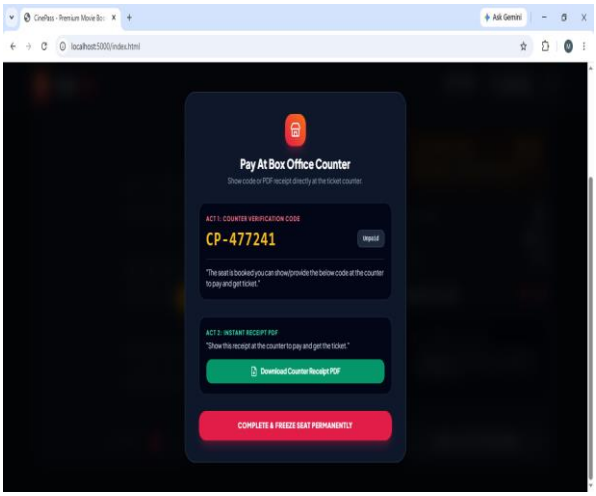
HOME PAGE



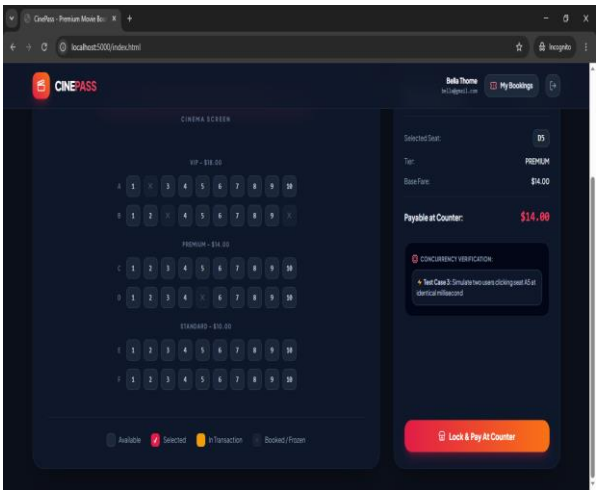
MY BOOKINGS



FOR LOCKING THE SEAT



ONE SEAT MULTIPLE BOOKING PAGE



LEARNING OUTCOMES:

1. Understand client-server communication using REST APIs and WebSockets for real-time applications.
2. Learn how to handle concurrency control and solve race conditions (Case 1, 2, and 3) in database transactions.
3. Master transaction window management using countdown timers and automated background database cleanup workers.
4. Learn to implement tokenless/session-based authentication and user dashboard tracking.
5. Understand client-side PDF document generation using jsPDF for digital receipts and ticketing workflows.

Maninder Kaur (25MCA20135)
25MCA-6 (B)